Типы данных для хранения денег

в финтех сервисах

и немного про C#, Go, JS/TS

@akrisanov

План обсуждения

- Типы данных: частые применения, плюсы и минусы использования
- Как работать с денежными значениями на бэкенде и в сервис-ориентированной архитектуре
- Типичные ошибки на фронтенде

В коде проектов можно встретить

1. **decimal** – представляет числа с фиксированным числом цифр после запятой/точки, что позволяет выполнять точные вычисления без ошибок округления; точность 28-29 цифр от $\pm 1.0*10^{-28}$ до $\pm 7.9228*10^{28}$
2. **int64** – целые 64-битные числа -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807
3. **uint64** (*ulong*) – беззнаковые целые числа 0 до 18 446 744 073 709 551 615
4. **float** – числа с плавающей запятой/точкой; точность ~6-9 цифр $-3.4*10^{38}$ до $3.4*10^{38}$

Минута интерактива

Какой тип данных по вашему стоит использовать?

Задачи в реальных проектах

- Хранение значений в БД и др. хранилищах
- Вычисления с деньгами
- Передача значений из одного сервиса в другой
- Отображение значений на фронтенде

Какие типы точно не использовать

- Float
- Double

Decimal и .NET

- Исторически “нужно работать с деньгами → используй Decimal”
  - Цены
  - Балансы
  - Внутренние счетчики: кол-во транзакций или товаров
  - Возвраты (refund)
  - НДС, % банка или сервиса, курсы валют и пр.
- Экосистема подталкивает разработчиков к выбору
  - SQL-схемы используют DECIMAL(18,2) → маппинг в EF Core (ORM)
  - ASP.NET модели хорошо работают с decimal
- Бизнес говорит: «Нам нужны числа с 2 числами после запятой»
- Недопонимание разницы между формой хранения данных и точностью вычислений

- Для **хранения и передачи значений** (строки или колонки в БД, события, вызовы API, файлы с дата сетами для анализа) на самом деле нужны целые числа с базовыми единицами (minor units):
  - Проще сравнение на равенство
  - Проще сравнение значений в целом и агрегация данных
  - Нет “шума” в логике связанного с округлениями
  - Проще взаимодействовать со сторонними системами, в особенности финансовыми

- Для **вычислений** (вклады, FX, и пр.) может потребоваться большая точность: *decimal* / *rational* / *bigInt*

А что в Go

- Нет встроенного decimal типа
- Но есть:
  - int (зависит от архитектуры) / int64
  - float32 / float64 → 🐛
  - math/big (big.Int, big.Rat, big.Float) → *перебор для большинства ситуаций*
  - Сторонние библиотеки → *усложнение кода и API*

```go
type Money int64 // minor units (cents, kopecks)

price := Money(1299)    // 12.99
balance := Money(10_000) // 100.00
```

Разделение ответственности в сервисах

- Хранилище и транспортный слой: БД, запросы, APIs
  - Используйте int64 для хранения сумм в базовых значениях (копейки, центы и пр.)
  - Маппируйте BIGINT в БД в int64 в коде
  - В событиях и сообщениях – всегда целые числа в базовых значениях
- Доменный слой
  - Свой тип данных / Value Object для ясности на основе int64 → *Money*
  - Логика и агрегации – суммы типа Money
  - Если нужны вычисления с % или курсами валют, переводим в decimal / big.Rat при вычислении и обратно
- Слой представления
  - Конвертируем Money в форматированную строку: “12.99”, “1 234,56 ₽”
  - Работает с валютами, локализацией, настройками пользователей

Когда реально нужен Decimal

- Interest rates: вклад, кредит, ипотека и т.д. → 3.75%
- FX rates: курсы валют → 1.234567 EUR/USD
- Per-day / per-year fees: комиссия за хранение → 0.25% в год
- Trading prices with 4-8+ decimals: криптовалюты, Форекс, акции и опционы → 1812.123456 USD

```go
// 1. Work in high precision decimal
rate := decimal.RequireFromString("0.0375") // 3.75%
amount := decimal.NewFromInt(balanceInCents).Div(decimal.NewFromInt(100))
interest := amount.Mul(rate)

// 2. Round and convert back to cents
interestCents := interest.Mul(decimal.NewFromInt(100)).Round(0)
money := Money(interestCents.IntPart())
```

В остальном – int64 для значений в центах

- Позволяет избежать тех же проблем с округлениями, что и Decimal
- 64-битный тип может хранить огромные значения безопасно → ~9e18 достаточно для хранения:
  - Мирового GDP в центах
  - Балансов, агрегированных значений, отчетов
  - Объемов торгов
  - Значений криптовалют
- Хорошо поддерживается в СУБД и хранилищах → BIGINT → нет необходимости приводить типы в коде
- Суммы могут быть отрицательными (возвраты, задолжности и пр.) → знаковый тип это покрывает

Платежные сервисы

- **Stripe** (триллионы оборота, мультивалютность)
  - Хранит суммы в int64 в копейках
  - На уровне кода оперирует своими явными типами на основе int64
  - В JSON { “amount”: 1299, “currency”: “usd” }
- **PayPal**
  - Внутри – всегда целые числа (long / int64)
  - На уровне API возвращает **строки**, например, “12.99”
- **GoCardless**
  - Целые числа (minor units) ← SEPA, BACS, округления не разрешены
- **Coinbase** (фиат и крипта)
  - Фиат хранят как Stripe и др.
  - Часто используют дополнительно int128 или big.Int (wei) в зависимости от криптовалюты
  - Все значения криптовалюты возвращаются в API в виде строк: “amount”: “0.00023456”

Кто часто использует Decimal внутри системы

- Банки и трейдинговые системы
  - FX-трейдинг
  - Производные инструменты (derivatives)
  - Начисление процентов
  - Амортизация кредитов
  - Ипотека
  - Казначейские системы
- Брокерские и инвестиционные платформы
  - Robinhood
  - Interactive Brokers
  - TradeRepublic
  - FTX / Coinbase (в отдельных частях — не для балансов, а для движков котировок)

- FX-движки и системы валютных конверсий
  - Wise, движок конверсий у Revolut
  - Спреды и котировки могут требовать:
    - 4 знака после запятой
    - иногда 6
    - иногда 8
- Крипто-трейдинговые движки
  - Криптовалюты имеют разную точность:
    - ETH: 18 знаков после запятой
    - SOL: 9
    - ADA: 6

Примеры из практики

Размер файла в килобайтах

```
public decimal SizeInKB { get; set; }
```

Размер файла в килобайтах — целое число, не требующее дробной части

`decimal` занимает 16 байт в памяти против 8 байт для `long`

Для размеров файлов не нужна высокая точность дробных значений — это счетная метрика

Диапазон `long (±9,223,372,036,854,775,807)` более чем достаточен для представления размеров файлов в КБ

Количество разрешений (лицензий)

```
public decimal Count { get; set; }  // Количественная
характеристика разрешения
```

Разрешения/лицензии — это всегда целые числа (1 пользователь, 5 пользователей, 10 счетов)

Невозможно иметь 2.5 пользователя или 3.7 счетов

Логика слияния разрешений работает с целыми числами:

```
result.Count = result.IsUnlimited ? 0 : (first.Count +
second.Count);
```

Денежные суммы хранятся как DECIMAL(18, 2)

DECIMAL(18, 2) = 18 цифр, 2 после запятой → Максимум: 9,999,999,999,999,999.99 (10 квадриллионов)

Реальная максимальная сумма в системе: ~миллионы/миллиарды рублей

```
-- Из БД
Value decimal(18, 2) NOT NULL
VatValue decimal(18, 2)
Sum decimal(18, 2)
TotalValue decimal(18, 2)
// Из C# кода
public decimal Value { get; set; }  // OperationPart – сумма операции
public decimal VatValue { get; set; }  // НДС
public decimal ValueInProductCurrency { get; set; }  // Цена товара
```

Проблемы с округлением всё равно есть

```
// Код полон округлений

decimal.Round(value, 2)

decimal.Round(totalValue * percent / 100, 2, MidpointRounding.ToEven)
```

Производительность БД

`DECIMAL(18,2)` занимает 9 байт в SQL Server

`BIGINT` занимает 8 байт и работает быстрее

Индексы на `DECIMAL` менее эффективны

Нет типа Money или Value Object

Вся бизнес-логика работает с голыми decimal:

```
decimal sumValue = valueInUserCurrency * Count * (100M - Discount) / 100M;
```

Проблемы с типобезопасностью:

```
// БЫЛО
decimal value1 = 100.50m;  // Что это? Рубли? Доллары? Копейки?
decimal value2 = 200.00m;
var sum = value1 + value2;  // Может сложить разные валюты!

// СТАЛО – безопасно
Money rub = Money.FromMajorUnits(100.50m, CurrencyEnum.RUB);
Money usd = Money.FromMajorUnits(200.00m, CurrencyEnum.USD);
var sum = rub + usd;  // ОШИБКА КОМПИЛЯЦИИ! Нельзя складывать разные валюты
```

Что можно сделать

Разделить decimal на категории и оставить Decimal для:

```
// 1. Курсы валют (много знаков после запятой)
public decimal CurrencyRate { get; set; }  // 74.3456789 RUB/USD

// 2. Проценты с дробной частью
public decimal InterestRate { get; set; }  // 3.75% годовых
public decimal Vat { get; set; }  // 20.5% НДС (хотя обычно целое)
public decimal Discount { get; set; }  // 15.5% скидка

// 3. Коэффициенты и множители
public decimal ExchangeMultiplier { get; set; }
```

Изменить типы на long (копейки)

```
// 1. Все денежные суммы
public long ValueInMinorUnits { get; set; }  // вместо decimal Value
public long VatValueInMinorUnits { get; set; }
public long TotalValueInMinorUnits { get; set; }

// 2. Цены в минимальных единицах
public long PriceInMinorUnits { get; set; }  // 12999 = 129.99 руб

// 3. Балансы счетов
public long AccountBalanceInMinorUnits { get; set; }
```

```
// БЫЛО (старый код)
public class OperationPartBase
{
    public decimal Value { get; set; }  // DECIMAL(18,2) в БД
    public decimal? VatValue { get; set; }
}

// СТАЛО
public class OperationPartModel
{
    // В БД: ValueInMinorUnits BIGINT NOT NULL
    private long _valueInMinorUnits;
    private int _currencyId;

    public Money Value => Money.FromMinorUnits(_valueInMinorUnits, _currencyId);

    public void SetValue(Money money)
    {
        _valueInMinorUnits = money.AmountInMinorUnits;
        _currencyId = money.CurrencyId;
    }
}

// Пример вычисления
var price = Money.FromMajorUnits(1000m, CurrencyEnum.RUB);  // 1000.00 руб
var withDiscount = price.ApplyDiscount(15m);  // 15% скидка = 850.00 руб
var withVat = withDiscount.AddVat(20m);  // + 20% НДС = 1020.00 руб
var total = withVat.Multiply(2.5m);  // × 2.5 шт = 2550.00 руб
```

А что насчет фронтенда?

Частая проблема – использование `number (float64)` для денежных сумм вместо библиотеки для работы с `Decimal` или `int64`.

Примеры:

- Расчет НДС
- Расчет скидок в счетах, инвойсах
- Расчет процентов при распределении операций
- Вычисления в бюджетах
- Проверка равенства денежных сумм
- Округления остатков на счетах
- Округления при экспорте в отчеты